Hazard Analysis

#	Hazard	Root cause (in this code)	Effect	Severity	Mitigation present
1	Missed button press	btn_sem is a binary semaphore — if it's already given and not yet taken, a second xSemaphoreGiveFromISR before the task drains it is silently lost (no count). Code comment flags this.	A real press produces no bottom-half action.	High if mapped to a safety-critical input (E-stop, alarm ack); low for a demo button	Not mitigated — wo counting semaphore presses must never b
2	Priority-inversion-like delay on the semaphore path	Both bottom-half tasks sit at equal priority (12); whichever the scheduler runs first can hold the CPU through its own blocking ESP_LOGI UART print, delaying the other from timestamping wake. Confirmed in data as the ~2192 µs cluster.	Reported/actual wake latency on one path balloons ~7x under otherwise-idle conditions, with no load tasks involved.	Medium — a false-idle-worst-case, not even the loaded scenario	Not mitigated in sca need distinct prioriti ESP_LOGI out of th critical path
3	UART mutex misuse from ISR	Scaffold correctly avoids this — printf/ESP_LOGI never called inside button_isr.	N/A (currently safe)	N/A	Already mitigated by logging is bottom-ha
4	Missing IRAM_ATTR (if induced)	ISR code lives in flash instead of always-mapped internal RAM. First interrupt after a quiescent period pays a cache-fill penalty.	First-press latency spikes 10–100x; later presses look normal, masking the defect on a quick spot-check.	High for anything requiring bounded worst-case latency	Mitigated in scaffold IRAM_ATTR; hazar if removed
5	Missing portYIELD_FROM_ISR (if induced)	Without it, a woken higher-priority task doesn't actually run until the next tick interrupt, even though it's ready.	Wake latency bounded below by the tick period (often 1–10 ms) regardless of signal speed.	High — turns an interrupt-driven system into a polled one in disguise	Mitigated in scaffold only if removed
6	xSemaphoreGive instead of ...FromISR (if induced)	Non-ISR-safe API called from interrupt context — touches scheduler state without ISR-safe bookkeeping.	Undefined behavior: corruption, crash, or silent misbehavior; not guaranteed to fail the same way twice.	High — safety-relevant because it's nondeterministic, not just wrong	Mitigated in scaffold FromISR variant
7	Debounce window too short/long	DEBOUNCE_US = 200 is tuned for Wokwi's clean simulated button.	Too short on real hardware → bounce reads as multiple presses; too long → legitimate fast presses dropped.	Medium, hardware-dependent	Scaffold flags this in needs re-tuning (~10 real button
8	Bottom-half tasks starved under load	Under WITH_LOAD=1, load Task A (priority 15) outranks both bottom-half tasks (12); it can run to completion before yielding, delaying wake.	Worst-case wake latency increases by however long Task A's remaining execution takes when a press lands mid-task.	Depends on Task A's WCET and how safety-critical the response is	Partially characterize priority assignment i hazard if this were a system

Note: items 1, 2, and 8 are latent characteristics of this scaffold's design as written, not just the induced failures in steps 4–6 — worth naming as known limitations in the README even if unaddressed.

Used Claude to create table